

Exercise 5: Configuring the Spring IoC Container

Source Code

BookRepository.java

```
package com.library.repository;

public class BookRepository {
    public void saveBook() {
        System.out.println("Book Repository is available.");
    }
}
```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayBook() {
        System.out.println("Book Service is working successfully.");
        bookRepository.saveBook();
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>
    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
</beans>
```

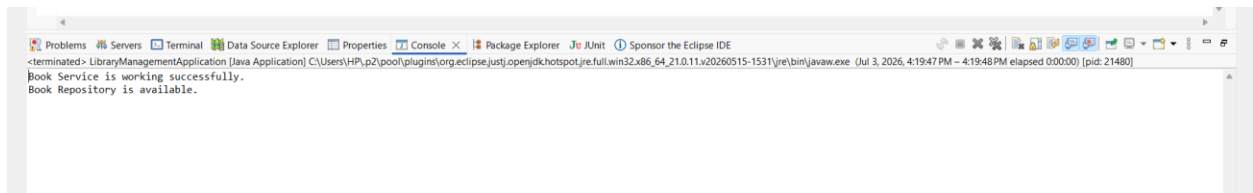
```
</bean>
</beans>
```

LibraryManagementApplication.java

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = context.getBean("bookService", BookService.class);
        service.displayBook();
    }
}
```

Output

Book Service is working successfully.
Book Repository is available.



Result

The Spring IoC Container was configured successfully using the applicationContext.xml file. The BookService and BookRepository beans were defined in the XML configuration file, and the BookRepository bean was injected into the BookService bean through setter injection. The application executed successfully and displayed the expected output, confirming that the Spring IoC Container was configured correctly.

Exercise 7: Implementing Constructor and Setter Injection

Source Code

BookRepository.java

```

package com.library.repository;

public class BookRepository {
    public void saveBook() {
        System.out.println("Book Repository is available.");
    }
}

```

BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private BookRepository bookRepository;
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void displayBook() {
        System.out.println("Book Service is working successfully.");
        bookRepository.saveBook();
    }
}

```

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="bookRepository"
        class="com.library.repository.BookRepository"/>
    <bean id="bookService"
        class="com.library.service.BookService">
        <constructor-arg ref="bookRepository"/>
        <property name="bookRepository"
            ref="bookRepository"/>
    </bean>
</beans>

```

```
</bean>
</beans>
```

LibraryManagementApplication.java

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = context.getBean("bookService", BookService.class);
        service.displayBook();
    }
}
```

Output

Book Service is working successfully.
Book Repository is available.



Result

Constructor injection and setter injection were implemented successfully using the Spring IoC container. The BookRepository bean was injected into the BookService bean through both constructor and setter injection using the XML configuration file. The application executed successfully and displayed the expected output, confirming that both dependency injection methods were configured correctly.

Exercise 9: Creating a Spring Boot Application

Source Code

Book.java

```
package com.library.entity;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Book {
    @Id
    private int id;
    private String title;
    private String author;
    public Book() {
    }
    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String author) {
        this.author = author;
    }
}
```

BookRepository.java

```
package com.library.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import com.library.entity.Book;
public interface BookRepository extends JpaRepository<Book, Integer> {
}
```

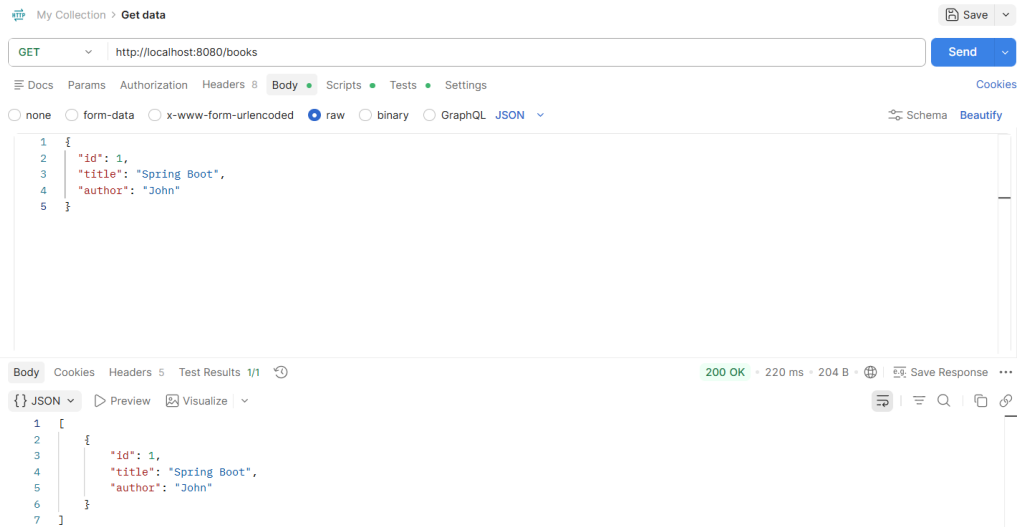
BookController.java

```
package com.library.controller;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.library.entity.Book;
import com.library.repository.BookRepository;
@RestController
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookRepository repository;
    @GetMapping
    public List<Book> getBooks() {
        return repository.findAll();
    }
    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }
}
```

application.properties

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
```

Output



Result

The Spring Boot application was created and configured successfully. The required dependencies were added, and the Book entity, BookRepository, and BookController classes were implemented. The application ran successfully, and the REST API was tested using Postman. The book details were added and retrieved successfully, confirming that the application was working as expected.